

Frontend Interview Questions & Answers

HTML • CSS • JavaScript • React

Student-Friendly Edition

LEVELS	STYLE
Easy • Medium • Advanced	Simple explanations + practical examples + interview tips

Prepared using current official documentation and standards-oriented sources. The explanations intentionally avoid unnecessary grammar and complexity so students can understand, remember, and explain the concepts in interviews.

Contents

HTML — 11 questions Easy: 8 | Medium: 3

CSS — 14 questions Easy: 4 | Medium: 10

JavaScript — 26 questions Easy: 7 | Medium: 10 | Advanced: 9

React — 20 questions Easy: 8 | Medium: 8 | Advanced: 4

Total: 71 interview questions. Every answer follows: simple meaning → practical explanation → small example → interview tip.

CODEGNAN
CODEGNAN

HTML

Structure & meaning of web documents

Easy

What is HTML?

HTML (HyperText Markup Language) is the standard markup language used to structure content on web pages. It tells the browser what each piece of content is: heading, paragraph, image, link, form, table, etc. HTML is not a programming language because it describes structure rather than performing calculations or logic.

```
<!doctype html>
<html>
  <body>
    <h1>Hello</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Interview Tip: Think of HTML as the skeleton of a webpage. CSS is the appearance and JavaScript is the behavior.

What is the current version of HTML?

Modern HTML is maintained as the HTML Living Standard rather than as a new numbered version such as 'HTML 6'. In interviews, saying 'HTML5' is still common because HTML5 introduced many important modern features, but the current standard is continuously updated.

Interview Tip: Best interview answer: 'HTML is currently maintained as the HTML Living Standard; HTML5 is the commonly used name for the modern HTML feature set.'

What is the role of <!DOCTYPE>?

The DOCTYPE declaration tells the browser which document mode to use. In modern HTML, <!DOCTYPE html> requests standards mode so the browser interprets the page using modern standards instead of legacy quirks behavior.

```
<!DOCTYPE html>
<html>...</html>
```

Interview Tip: DOCTYPE is a declaration, not an HTML element.

What is semantics in HTML?

Semantic HTML uses elements according to their meaning, not just their visual appearance. For example, <nav> represents navigation, <main> represents the main content, and <article> represents a self-contained piece of content. Semantic markup improves structure, accessibility, and maintainability.

```
<header>Site Header</header>
<nav>Menu</nav>
<main>
  <article>News article</article>
</main>
<footer>Copyright</footer>
```

Interview Tip: Use a semantic element when its meaning matches the content; use <div> when no suitable semantic element exists.

What is the role of alt in an image?

The alt attribute provides alternative text for an image. Screen readers can announce it, and it is shown/used as a text alternative when the image cannot be displayed. Decorative images can usually use alt="" so assistive technology can ignore them.

```

```

Interview Tip: Write alt text based on the purpose of the image, not a long description of every visual detail.

What are meta tags?

Meta elements provide information about the document that is not normally displayed as page content. Common examples include character encoding, viewport settings, description, and other document metadata.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Frontend interview notes">
```

Interview Tip: The viewport meta tag is especially important for responsive mobile pages.

What are block and inline elements?

A block-level box normally starts on a new line and can take available horizontal space. An inline box normally flows within a line of text and takes only the space it needs. CSS display rules can change how elements behave, so think of this as the default layout behavior rather than a permanent HTML classification.

```
<div>Block</div>
<span>Inline</span>
<a href="#">Inline link</a>
```

Interview Tip: Modern CSS lets you change an element's display behavior with properties such as block, inline, inline-block, flex, and grid.

What is the difference between <div> and ?

<div> is a generic container normally used for grouping larger/block content. is a generic inline container normally used for grouping or styling a small part of text or inline content. Neither adds semantic meaning by itself.

```
<div class="card">
  <h2>Doctor</h2>
</div>

<p>Hello <span class="highlight">Teja</span></p>
```

Medium

How can we do internal routing in HTML?

Traditional HTML internal navigation uses anchor links that point to an element ID on the same page. This is page-section navigation, not SPA routing.

```
<nav>
  <a href="#about">About</a>
  <a href="#contact">Contact</a>
</nav>

<section id="about">About us</section>
<section id="contact">Contact us</section>
```

Interview Tip: For React applications, client-side routing is normally handled with a routing library such as React Router.

What are GET and POST methods in forms?

GET sends form data as URL query parameters, for example /search?q=react. It is suitable for requests that retrieve data and do not cause a server-side change. POST sends form data in the request body and is commonly used when submitting data that creates or changes server-side state.

```
<form action="/search" method="get">
  <input name="q">
  <button>Search</button>
</form>

<form action="/users" method="post">
  <input name="name">
  <button>Create</button>
</form>
```

Interview Tip: GET is visible in the URL; POST data is normally carried in the request body. HTTPS is still required for sensitive data.

What are some important form attributes?

Common form attributes include action, method, enctype, autocomplete, novalidate, target, and name. action specifies where data is sent; method selects GET or POST; enctype controls how the body is encoded, especially for file uploads; autocomplete controls browser autofill behavior; novalidate disables built-in constraint validation.

```
<form
  action="/register"
  method="post"
  autocomplete="on">
  <input name="name" required>
  <button type="submit">Register</button>
</form>
```

Interview Tip: For file uploads, use method="post" and enctype="multipart/form-data".

CODEGNAN
CODEGNAN

CSS

Styling, layout, responsive design & motion

Easy

What is CSS?

CSS (Cascading Style Sheets) controls the presentation and layout of HTML. It is used for colors, fonts, spacing, sizing, positioning, responsive layouts, transitions, and animations.

```
h1 { color: blue; }
```

Interview Tip: HTML gives structure; CSS controls presentation.

How can we insert CSS?

There are three common ways: inline CSS using the style attribute, internal CSS using a <style> element, and external CSS using a separate .css file linked with <link>. External CSS is usually preferred for maintainability.

```
<!-- Inline -->
<p style="color:red">Hello</p>

<!-- Internal -->
<style>
  p { color: red; }
</style>

<!-- External -->
<link rel="stylesheet" href="style.css">
```

Explain the CSS box model.

Every element is laid out as a box. The main parts are content, padding, border, and margin. Padding is space inside the border around the content; margin is space outside the border.

```
/* easier sizing */
* {
  box-sizing: border-box;
}

.card {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 20px;
}
```

Interview Tip: With border-box, the declared width includes content + padding + border.

What is display: inline-block?

inline-block combines useful parts of inline and block behavior. Multiple elements can sit on the same line like inline elements, while the element can still accept width, height, padding, and margin more like a block-level box.

```
.item { display: inline-block; width: 150px; height: 80px; }
```

Medium

Explain units in CSS.

CSS units can be absolute or relative. Common absolute unit: px. Common relative units: %, em, rem, vw, vh, and fr for grid tracks. rem is relative to the root font size; em is relative to the relevant font size; % often depends on the containing context; vw/vh are viewport units; fr distributes available grid space.

```
h1 { font-size: 2rem; }
.box { width: 50%; }
.hero { min-height: 100vh; }
.grid { grid-template-columns: 1fr 2fr; }
```

Interview Tip: For responsive layouts, prefer flexible units and layout systems instead of making every size a fixed pixel value.

What are the types of CSS position?

The main position values are static, relative, absolute, fixed, and sticky. static is the normal flow. relative keeps the element in normal flow but provides an offset reference for itself and often for absolutely positioned descendants. absolute is removed from normal flow and positioned relative to a containing block. fixed is positioned relative to the viewport in typical use. sticky behaves like relative until a scroll threshold is reached, then sticks within its scroll container.

```
parent { position: relative; }
.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

Interview Tip: When an absolute child should be positioned against a parent, a common pattern is parent: position: relative and child: position: absolute.

What is z-index?

z-index controls stacking order when elements overlap in a relevant stacking context. A larger z-index can place an element above another element, but stacking contexts can affect the result.

```
.modal { position: fixed; z-index: 1000; }
.header { position: relative; z-index: 10; }
```

Interview Tip: z-index does not simply beat every other z-index on the page; stacking contexts matter.

What is Flexbox?

Flexbox is a one-dimensional CSS layout system. It arranges items mainly in a row or a column and provides simple controls for alignment, spacing, growing, shrinking, and wrapping.

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Interview Tip: Use Flexbox when the main layout problem is one-dimensional.

What are Flexbox parent and child properties?

Parent/container properties include display:flex, flex-direction, flex-wrap, justify-content, align-items, align-content, and gap. Child/item properties include order, flex-grow, flex-shrink, flex-basis, flex, align-self, and sometimes margin:auto for alignment.

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.item {
  flex: 1 1 200px;
}
```

What is CSS Grid?

CSS Grid is a two-dimensional layout system for rows and columns. It is useful for page layouts, card grids, dashboards, and designs where both dimensions matter.

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

Interview Tip: Use Grid when you need to reason about rows and columns together.

What are Grid parent and child properties?

Grid container properties include `display:grid`, `grid-template-columns`, `grid-template-rows`, `gap`, `grid-template-areas`, `justify-items`, `align-items`, `justify-content`, `align-content`, and `auto-placement` properties. Grid item properties include `grid-column`, `grid-row`, `grid-area`, `justify-self`, `align-self`, and `place-self`.

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.feature {
  grid-column: 1 / 3;
}
```

What are media queries?

Media queries let CSS apply rules conditionally based on the environment, commonly viewport width, but also features such as orientation or user preferences. They are widely used for responsive design.

```
@media (max-width: 768px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```

Interview Tip: A mobile-first approach often starts with the narrow layout and adds rules for wider screens.

What is the difference between transition and transform?

`transform` changes an element's geometry or visual coordinate system, such as `translate`, `scale`, `rotate`, or `skew`. `transition` controls how a property changes over time between states. They are often used together: `transform` creates the movement/scale and `transition` makes it smooth.

```
.card {
  transition: transform 0.3s ease;
}

.card:hover {
  transform: translateY(-5px) scale(1.02);
}
```

How do you add animations in CSS?

Define animation stages with `@keyframes` and apply them with animation properties such as `animation-name`, `animation-duration`, `animation-timing-function`, `animation-delay`, `animation-iteration-count`, `animation-direction`, and `animation-fill-mode`.

```
@keyframes slideIn {
  from { transform: translateX(-30px); opacity: 0; }
  to   { transform: translateX(0); opacity: 1; }
}

.card {
  animation: slideIn 0.6s ease;
}
```

Interview Tip: Respect `prefers-reduced-motion` when animations are not essential.

JavaScript

Language fundamentals, browser behavior & asynchronous programming

Easy

What is JavaScript?

JavaScript is a programming language used to add logic and behavior to web applications. In the browser it can respond to events, manipulate the DOM, validate forms, communicate with APIs, and update the UI. It is also used outside the browser, for example with Node.js.

```
button.addEventListener('click', () => alert('Hello'));
```

Difference between var, let, and const.

var is function-scoped and can be redeclared; let and const are block-scoped. let can be reassigned, while const cannot be reassigned after initialization. let and const are generally preferred in modern JavaScript.

```
var a = 10;
let b = 20;
const c = 30;
```

```
b = 40; // allowed
// c = 50; // error
```

Interview Tip: const prevents reassignment of the variable binding; it does not make an object or array deeply immutable.

Difference between == and ===.

== is loose equality and may perform type conversion before comparing. === is strict equality and compares both type and value without the usual implicit coercion. In most application code, === is preferred when you want predictable comparisons.

```
5 == "5" // true
5 === "5" // false
```

What is the DOM?

DOM stands for Document Object Model. The browser represents the HTML document as a tree of objects. JavaScript can use DOM APIs to find elements, change content or styles, create elements, and attach event listeners.

```
const title = document.getElementById('title');
title.textContent = 'Welcome';
```

How do you stop a form from automatically submitting?

Listen for the submit event and call event.preventDefault(). This stops the browser's default form submission behavior so JavaScript can validate data or send it with fetch/Axios.

```
form.addEventListener('submit', event => {
  event.preventDefault();
  // custom logic
});
```

What is the role of the spread operator?

The spread syntax (...) expands an iterable into individual values or copies enumerable properties into a new object. It is commonly used to copy arrays/objects and to create updated state without mutating the original outer structure.

```
const a = [1, 2];
const b = [...a, 3];

const user = { name: 'A', age: 20 };
const updated = { ...user, age: 21 };
```

What is destructuring?

Destructuring is syntax for extracting values from arrays or properties from objects into variables.

```
const user = {
```

```

    name: 'Teja',
    age: 25
  };

  const { name, age } = user;

  const numbers = [10, 20];
  const [first, second] = numbers;

```

Medium

What is hoisting?

Hoisting describes how JavaScript processes declarations before executing code in a scope. Function declarations can generally be called before their declaration. `var` declarations are initialized as `undefined` before execution reaches their assignment. `let` and `const` are also hoisted in the specification sense but cannot be accessed before initialization because they are in the temporal dead zone.

```

console.log(a); // undefined
var a = 10;

sayHello(); // works
function sayHello() {
  console.log('Hello');
}

```

What are the types of scope in JavaScript?

Common scope categories are global scope, function scope, block scope, and module scope. `var` is function-scoped, while `let` and `const` are block-scoped. A function can access variables from its outer lexical scopes.

```

let globalValue = 10;

function demo() {
  let local = 20;

  if (true) {
    let blockValue = 30;
  }
}

```

What are the types of functions in JavaScript?

Common forms include function declarations, function expressions, arrow functions, anonymous functions, immediately invoked function expressions (IIFE), callback functions, and higher-order functions. These categories can overlap: a function can be an arrow function and also be used as a callback.

```

function add(a, b) {
  return a + b;
}

const multiply = function(a, b) {
  return a * b;
};

const square = n => n * n;

```

What are synchronous and asynchronous operations?

Synchronous code runs step by step and the next operation waits for the previous one to finish. Asynchronous operations allow other JavaScript work to continue while waiting for something such as a timer, network response, or other external operation.

```

console.log('A');

setTimeout(() => {
  console.log('C');
}, 0);

console.log('B');

// A
// B

```

```
// c
```

What is a callback function?

A callback is a function passed to another function so it can be called later. Callbacks are common in event handling, array methods, and asynchronous APIs.

```
function greet(name, callback) {  
  callback('Hello ' + name);  
}  
  
greet('Teja', message => {  
  console.log(message);  
});
```

What is a Promise?

A Promise represents the eventual result of an asynchronous operation. It can be pending, fulfilled, or rejected. Promises make asynchronous success and failure easier to compose using then, catch, and finally.

```
const promise = fetch('/users');  
  
promise  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```

How do you consume a Promise?

The common methods are .then() for a fulfilled result, .catch() for rejection, and .finally() for code that should run after completion regardless of success or failure. You can also consume a Promise with async/await.

```
getData()  
  .then(data => console.log(data))  
  .catch(error => console.error(error))  
  .finally(() => console.log('Done'));
```

What is async/await?

async/await is syntax built on top of Promises. An async function returns a Promise. await pauses that async function until the awaited Promise settles, making asynchronous code easier to read without blocking the entire JavaScript program.

```
async function loadUsers() {  
  try {  
    const response = await fetch('/users');  
    const users = await response.json();  
    console.log(users);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Explain map, filter, and reduce.

These are common higher-order array methods. map creates a new array by transforming each item. filter creates a new array containing items that pass a condition. reduce combines an array into one result such as a sum, object, or grouped structure.

```
const prices = [100, 200, 300];  
  
const doubled = prices.map(p => p * 2);  
const expensive = prices.filter(p => p > 150);  
const total = prices.reduce((sum, p) => sum + p, 0);
```

How do you call an API in JavaScript?

The browser provides fetch for HTTP requests. You normally await the response, check whether the request succeeded, and then parse the body such as JSON. Libraries such as Axios provide another API for the same job.

```
async function getUsers() {  
  const response = await fetch('https://example.com/users');  
  
  if (!response.ok) {
```

```

    throw new Error('Request failed');
  }

  const data = await response.json();
  return data;
}

```

Advanced

What is a closure?

A closure is a function together with access to variables from its surrounding lexical scope. The inner function can continue to use those variables even after the outer function has finished.

```

function counter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const next = counter();
next(); // 1
next(); // 2

```

Interview Tip: A common real-world use is keeping private state inside a function.

What is shallow copy vs deep copy?

A shallow copy creates a new outer object/array but nested objects are still shared references. A deep copy creates independent nested data. Common shallow-copy tools include spread and Object.assign. structuredClone can deep-clone many structured-clone-compatible values.

```

const original = {
  name: 'A',
  address: { city: 'Vizag' }
};

const shallow = { ...original };
shallow.address.city = 'Hyderabad';
// original.address.city is also changed

const deep = structuredClone(original);

```

Interview Tip: Do not describe JSON.parse(JSON.stringify(...)) as a universal deep-copy solution; it loses or changes some value types.

What is the event loop?

The event loop is part of the browser's JavaScript execution model. JavaScript executes synchronous work on the call stack. When asynchronous work is ready, its callbacks are queued. The event loop coordinates when queued work can be moved to the stack after the current synchronous work finishes. Promise reactions use the microtask queue, which is processed before the browser moves on to the next task in the usual event-loop model.

```

console.log('A');

setTimeout(() => console.log('B'), 0);

Promise.resolve().then(() => console.log('C'));

console.log('D');

// A
// D
// C
// B

```

What is the this keyword?

this refers to a value determined by how a function is called. In a method call, it commonly refers to the object before the dot. With new, it refers to the newly created object. Arrow functions do not create their own this; they capture it lexically from the surrounding scope. Strict mode also changes some default this behavior.

```
const user = {
  name: 'Teja',
  show() {
    console.log(this.name);
  }
};

user.show(); // Teja
```

Interview Tip: For interview questions, always explain this using the call site rather than saying simply 'this means the current object.'

What are call, apply, and bind?

call and apply invoke a function immediately while letting you choose its this value. call receives arguments separately; apply receives arguments as an array-like value. bind does not immediately invoke the function; it returns a new function with this and optionally initial arguments fixed.

```
function greet(city) {
  console.log(this.name, city);
}

const user = { name: 'Teja' };

greet.call(user, 'Vizag');
greet.apply(user, ['Vizag']);

const bound = greet.bind(user, 'Vizag');
bound();
```

What is event bubbling and event capturing?

When an event occurs on a nested element, it travels through the DOM. Capturing travels from the outer ancestors toward the target; at the target it runs target handling; bubbling travels from the target back toward ancestors. Event listeners use bubbling by default unless capture is enabled.

```
parent.addEventListener('click', handler);
parent.addEventListener('click', handler, { capture: true });
```

Interview Tip: event.stopPropagation() can stop further propagation, but use it carefully because it can interfere with other handlers.

What is memoization?

Memoization is an optimization technique that caches the result of a function for a given input so repeated calls can reuse previous work. It is useful when a calculation is expensive and inputs repeat.

```
function memoize(fn) {
  const cache = new Map();

  return value => {
    if (cache.has(value)) return cache.get(value);
    const result = fn(value);
    cache.set(value, result);
    return result;
  };
}
```

Interview Tip: Caching uses memory, so memoization should be used when the saved computation is worth the extra storage.

What is debouncing?

Debouncing delays a function until a quiet period has passed after the last event. If events keep arriving, the timer is reset. It is useful for search boxes, where you may want to wait until the user pauses typing before calling an API.

```
function debounce(fn, delay) {
```

```
let timer;

return (...args) => {
  clearTimeout(timer);
  timer = setTimeout(() => fn(...args), delay);
};
}
```

What is throttling?

Throttling limits how often a function can run during a period of continuous events. It is useful for high-frequency events such as scroll, resize, or mouse movement when you want regular updates rather than one final call.

```
function throttle(fn, delay) {
  let allowed = true;

  return (...args) => {
    if (!allowed) return;

    fn(...args);
    allowed = false;

    setTimeout(() => {
      allowed = true;
    }, delay);
  };
}
```

Interview Tip: Easy memory trick: debounce = wait for silence; throttle = limit the rate.

CODEGNAN
CODEGNAN

React

Components, Hooks, routing, state & performance

Easy

What is React and what are its main features?

React is a JavaScript library for building user interfaces from reusable components. Its common concepts include component-based UI, JSX, props, state, declarative rendering, hooks, and efficient update/reconciliation behavior. React itself is focused on UI; routing, HTTP clients, and global state are commonly handled with additional libraries.

```
function Welcome({ name }) {  
  return <h1>Hello {name}</h1>;  
}
```

Interview Tip: Think: React lets you describe what the UI should look like for the current data.

What are the types of components and their differences?

Modern React primarily uses function components. Older React applications may contain class components. Function components are JavaScript functions that return JSX and can use Hooks. Class components extend `React.Component` and use lifecycle methods and class state. For new development, function components with Hooks are the normal approach.

```
function Welcome() {  
  return <h1>Hello</h1>;  
}  
  
// Older class style:  
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

What is state in React?

State is data a component remembers and that can change over time. When state changes through its setter, React schedules a re-render so the UI can reflect the new state.

```
const [count, setCount] = useState(0);  
  
<button onClick={() => setCount(count + 1)}>  
  {count}  
</button>
```

Interview Tip: Do not directly mutate state. Use the state setter.

What are props?

Props are inputs passed from a parent component to a child component. They are read-only from the child's point of view and are useful for passing data and callback functions.

```
function DoctorCard({ name, salary }) {  
  return <p>{name} - {salary}</p>;  
}  
  
<DoctorCard name="Teja" salary={50000} />
```

What is useState?

`useState` is a React Hook that adds a state variable to a function component. It returns the current state value and a setter function.

```
const [name, setName] = useState('');  
  
setName('Teja');
```

Interview Tip: The setter schedules an update; it should not be treated as a normal synchronous assignment.

What is Axios?

Axios is an HTTP client library commonly used in React applications to call APIs. It supports GET, POST, PUT/PATCH, DELETE, request configuration, headers, interceptors, and convenient response handling.

```
import axios from 'axios';

const response = await axios.get('/api/users');
console.log(response.data);
```

Interview Tip: Axios is not part of React itself; it is a separate library.

How do you render a list in React?

Use JavaScript array methods such as map to turn each data item into JSX.

```
const doctors = ['Teja', 'Sam', 'Anu'];

return (
  <div>
    {doctors.map(doctor => (
      <p key={doctor}>{doctor}</p>
    ))}
  </div>
);
```

What are React Fragments?

Fragments let a component return multiple sibling elements without adding an extra DOM wrapper.

```
return (
  <>
    <h1>Doctors</h1>
    <p>List of doctors</p>
  </>
);
```

Interview Tip: The short syntax <>...</> cannot receive a key; use <React.Fragment key={...}> when a keyed fragment is needed.

Medium

What is useEffect?

useEffect lets a component synchronize with external systems such as network requests, browser APIs, subscriptions, timers, or third-party widgets. It runs after rendering according to its dependency list and can return a cleanup function.

```
useEffect(() => {
  fetchUsers();

  return () => {
    // cleanup if needed
  };
}, []);
```

Interview Tip: Do not use useEffect for every calculation. If something can be derived directly during rendering, an Effect may not be needed.

What is controlled form handling in React?

In a controlled input, React state is the source of truth for the input value. The value prop displays the state and onChange updates the state.

```
const [name, setName] = useState('');

<input
  value={name}
  onChange={e => setName(e.target.value)}
/>
```

Interview Tip: Controlled forms make validation and form submission logic easier to coordinate.

What is the role of the key attribute?

A key gives React a stable identity for each item in a rendered list. React uses keys to understand which items were added, removed, or moved and to preserve the right component instances. Keys should be stable and unique among siblings.

```
users.map(user => (  
  <UserCard key={user.id} user={user} />  
))
```

Interview Tip: Avoid using array indexes as keys when list order can change or items can be inserted/removed.

What is React Router DOM?

React Router is used to manage client-side navigation and route matching in React applications. It lets an SPA display different UI for different URLs without requiring a full document reload for every navigation.

```
<Routes>  
  <Route path="/" element={<Home />} />  
  <Route path="/doctors" element={<Doctors />} />  
</Routes>
```

Interview Tip: Current React Router documentation uses the react-router package APIs; many React projects still use react-router-dom depending on the routing setup/version.

What are useNavigate and useParams?

useNavigate returns a function for programmatic navigation. useParams reads dynamic parameters matched from the current route.

```
const navigate = useNavigate();  
navigate('/doctors');
```

```
const { id } = useParams();  
// route: /doctors/:id
```

What is useContext?

useContext lets a component read a value from a React Context without manually passing that value through every intermediate component. It is useful for values such as theme, authenticated user information, or application-level settings.

```
const ThemeContext = createContext('light');  
  
function Button() {  
  const theme = useContext(ThemeContext);  
  return <button>{theme}</button>;  
}
```

Interview Tip: Context is not automatically a replacement for every global state solution; choose it based on how frequently and broadly data changes.

What is props drilling?

Props drilling means passing data through several intermediate components even when those components do not need the data, only so a deeper component can receive it.

```
App  
  ■■ Parent  
    ■■ Child  
      ■■ GrandChild ← needs user
```

Interview Tip: Possible solutions include component composition, Context, or a state-management library, depending on the problem.

What is state lifting (lifting state up)?

Lifting state up means moving shared state to the nearest common parent of the components that need it. The parent owns the state and passes the value and update callbacks to its children.

```
function Parent() {  
  const [search, setSearch] = useState('');
```

```

    return (
      <>
        <Search value={search} onChange={setSearch} />
        <DoctorList search={search} />
      </>
    );
  }
}

```

Interview Tip: The goal is one clear source of truth for shared data.

Advanced

How can you optimize a React application?

Start by measuring the actual problem. Common techniques include keeping components pure, avoiding unnecessary state, deriving values instead of storing duplicates, splitting large components, using stable keys, lazy-loading large routes/components, and using memoization tools such as `memo`, `useMemo`, and `useCallback` when they prevent meaningful repeated work. React Compiler can also optimize some components automatically in supported setups.

```

const visible = useMemo(
  () => filterDoctors(doctors, search),
  [doctors, search]
);

const Card = memo(DoctorCard);

```

Interview Tip: Do not add `useMemo`/`useCallback` everywhere. Optimization should solve a measured or clear performance problem.

What is Redux Toolkit?

Redux Toolkit (RTK) is the recommended official way to write Redux logic. It simplifies store setup, reducers, immutable updates, slices, middleware, and common patterns. In React, `react-redux` provides bindings such as `Provider`, `useSelector`, and `useDispatch`.

```

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => {
      state.value += 1;
    }
  }
});

```

Interview Tip: RTK reduces the boilerplate of older handwritten Redux patterns.

What is a Pure Component?

A pure React component is one whose rendering is predictable: the same inputs should produce the same output and render should avoid side effects. In class components, `React.PureComponent` performs a shallow comparison of props and state to skip some renders. In function components, `React.memo` can skip re-rendering when props are unchanged according to its comparison.

```

const UserCard = memo(function UserCard({ name }) {
  return <p>{name}</p>;
});

```

Interview Tip: Purity is a React rule; memoization is an optimization technique built on the assumption that rendering is pure.

What are lazy loading and Suspense?

`React.lazy` lets you defer loading a component's code until it is rendered. `Suspense` provides a fallback UI while that lazy component is loading. This can reduce the initial JavaScript needed for a page, especially when routes or large features are split into separate chunks.

```

const Reports = lazy(() => import('./Reports'));

<Suspense fallback=<p>Loading...</p>>
  <Reports />
</Suspense>

```

Interview Tip: A common practical use is route-level code splitting.

CODEGNAN
CODEGNAN

Sources & Further Reading

MDN Web Docs — HTML

<https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs — CSS

<https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs — JavaScript Guide

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

React Official Documentation

<https://react.dev/>

React Router Documentation

<https://reactrouter.com/>

Redux Toolkit Documentation

<https://redux-toolkit.js.org/>

Important note: Web standards and libraries evolve. HTML is maintained as a Living Standard, and React/React Router APIs can change across releases. Verify version-specific behavior in official documentation when a project depends on a particular version.

CODEGNAN
CODEGNAN